

Dynamic Programming

Floyd-Warshall Algorithm

All-pairs shortest paths

Applying Bellman-Ford for all vertices as the starting node will cost $O(|V|^2|E|)$

A DP solution in $O(|V|^3)$!

the shortest path $u \rightarrow w_1 \rightarrow \dots \rightarrow w_l \rightarrow v$ between u and v

Subproblem:

Lets start with shortest paths between all pairs not involving any intermediate nodes, and grow from there considering 1 node in the graph at a time till we consider all.

Vertices: $\{1, 2, \dots, n\}$

$\text{dist}(i, j, k)$ 

The shortest path from i to j considering only the nodes $\{1, 2, \dots, k\}$ as the intermediates

Base case:

$\text{dist}(i, j, 0) \longrightarrow$ If an edge exist between i and j , then it is it is length $l(i, j)$ [0 for $i = j$] or it is ∞ .

Assuming no negative cycles, with the introduction of the node k into the set of intermediate nodes being considered $\{1, \dots, k - 1\}$, we need to recalculate all distance values!

k gives us a shorter path from i to j if and only if

$$\text{dist}(i, k, k - 1) + \text{dist}(k, j, k - 1) < \text{dist}(i, j, k - 1)$$

$\text{dist}(i, j, k) \longleftarrow$

for $i = 1$ to n :

for $j = 1$ to n :

$\text{dist}(i, j, 0) = \infty \rightarrow 0, i=j$

for all $(i, j) \in E$:

$\text{dist}(i, j, 0) = l(i, j)$

for $k = 1$ to n :

for $i = 1$ to n :

for $j = 1$ to n :

$\text{dist}(i, j, k) = \min\{\text{dist}(i, k, k - 1) + \text{dist}(k, j, k - 1), \text{dist}(i, j, k - 1)\}$

Bookkeeping (store min path)

$O(n^3)$ time complexity

Memory: TD - $O(n^3)$, BU - $O(n^2)$

NP-Complete & NP-Hard

Introduction

Saw efficient POLYNOMIAL TIME algorithms for finding a shortest path in a graph, a graph's minimum spanning tree, etc.

They are intelligent ways of reducing the search space of solutions!

A graph with n vertices can have upto n^{n-2} spanning trees (complete graph) [Kirchhoff's /Cayley's Theorem], and a typical graph has an exponential number of paths from s to t (complete: $(n-2)!$).

So exhaustive search would be of exponential (or worse) complexity

But for certain problems, the most efficient solution invented till now is still of exponential complexity, may not much simpler than exhaustive search.

Satisfiability (SAT)

clauses *literals*

AND OR

$(x \vee y \vee z) (x \vee \overline{y}) (y \vee \overline{z}) (z \vee \overline{x}) (\overline{x} \vee \overline{y} \vee \overline{z})$

Boolean formula in conjunctive normal form (CNF)

Either find an assignment such that the result is TRUE or report that it does not exist. $\longrightarrow$ *satisfying truth assignment*

3-SAT (problem with max 3 literals in a clause, 1&2-SATs are Polynomial)

n variables in the expression means 2^n possibilities, and an exhaustive search through them to perform the task on all clauses.


search problem

instance I

solution S

A search problem must have the property that any proposed solution S to an instance I can be quickly checked for correctness



A *search problem* is specified by an algorithm $\mathcal{C}$ that takes two inputs, an instance I and a proposed solution S , and runs in time polynomial in $|I|$. We say S is a solution to I if and only if $\mathcal{C}(I, S) = \text{true}$.



Input size

Travelling Salesman Problem (TSP)

Seeing TSP as a search problem. (slightly different!)

n vertices $1, \dots, n$ $n(n-1)/2$ distances between them

budget b

Check whether: $d_{\tau(1),\tau(2)} + d_{\tau(2),\tau(3)} + \dots + d_{\tau(n),\tau(1)} \leq b.$

A permutation $\tau(1), \dots, \tau(n)$ of the vertices $(n-1)!$

Exhaustive search would be factorial! Our DP TSP algorithm was polynomial x exponential!

find the optimum tour and if it is within budget,
return it; if not, there is no solution

P and NP

Non-deterministic* polynomial time

We denote the class of all search problems by **NP**

can be quickly checked for correctness

$\mathcal{C}(I, S)$

running time bounded by a polynomial in $|I|$

*The class of all search problems that can be solved in polynomial time is denoted **P**. (Deterministic*)*

Are there search problems that cannot be solved in polynomial time?

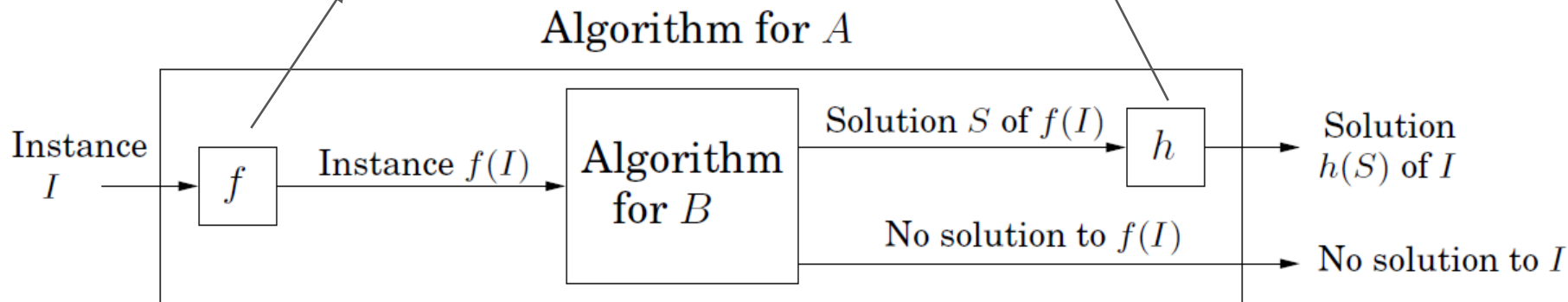
P $\neq$ **NP**?

Most experts think so, but not proven.

* Turing machine

Reduction: Polynomial time

Polynomial time



The converse may not be true!

$$A \longrightarrow B$$

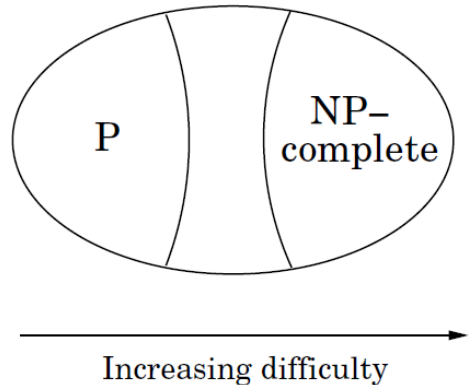
If $A \longrightarrow B$ and $B \longrightarrow C$, then $A \longrightarrow C$

If A is hard then B is also hard, if B is efficient so is A .

A problem is **NP**-hard if all search problems in **NP** reduce to it. If a **NP**-hard problem is in **NP** (or is a search problem), then it is **NP**-complete.

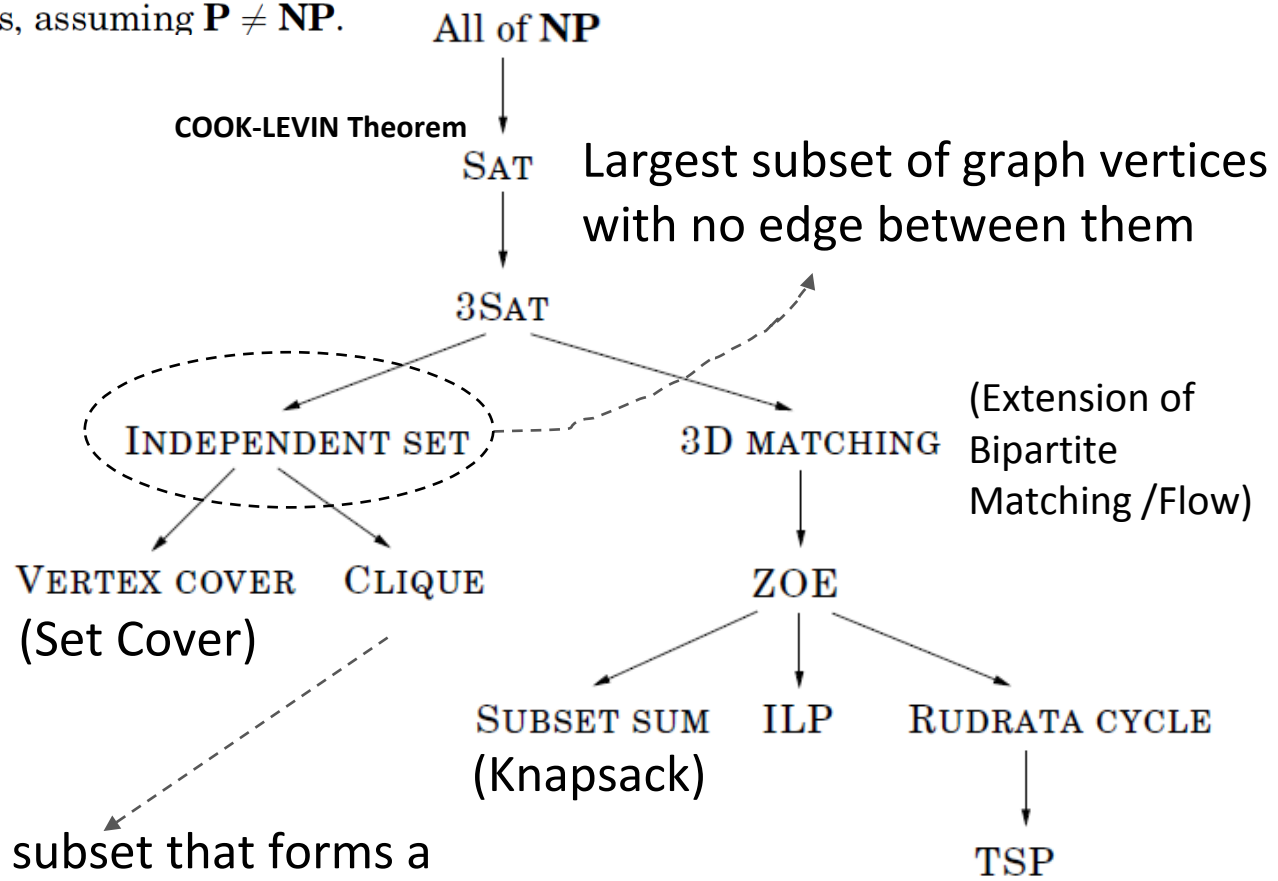
If even one **NP**-complete problem is in **P**, then **P** = **NP**. Note: **P** $\subseteq$ **NP**

The space **NP** of all search problems, assuming **P** ≠ **NP**.



Karp's 21 NP-complete problems!

Largest vertex subset that forms a complete graph (all edges present)



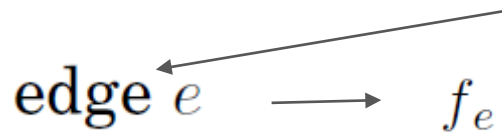
Network Flow

Flow in Networks

Goal: A directed graph. Send as much quantity possible from source s to sink t . There is a maximum capacity associated with every edge in the graph.

The networks we are dealing with consist of a directed graph $G = (V, E)$; two special nodes $s, t \in V$, which are, respectively, a source and sink of G ; and *capacities* $c_e > 0$ on the edges.

Such a scheme is called a *Flow* (Network Flow) $G = (V, E)$



1. It doesn't violate edge capacities: $0 \leq f_e \leq c_e$ for all $e \in E$.
2. For all nodes u except s and t , the amount of flow entering u equals the amount leaving

* Feasible flow

u :

$$\sum_{(w,u) \in E} f_{wu} = \sum_{(u,z) \in E} f_{uz}.$$

flow is *conserved*

size of a flow $\longrightarrow$ $\text{size}(f) = \sum_{(s,u) \in E} f_{su}.$

Task: Find $\{f_e : e \in E\}$

Given a set of linear constraints and a linear objective function to maximize!

Linear Program! Max-flow problem is a linear programming issue.

maximize $f_{sa} + f_{sb} + f_{sc}$

27 constraints

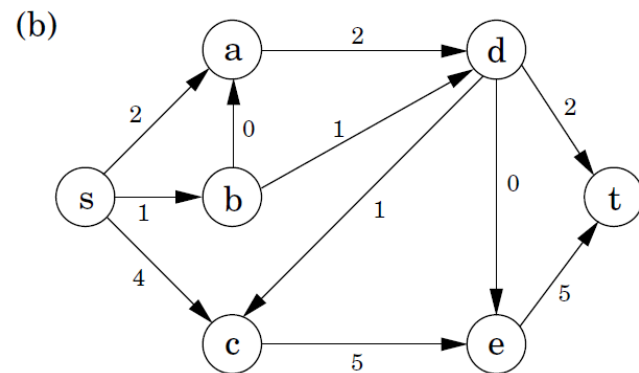
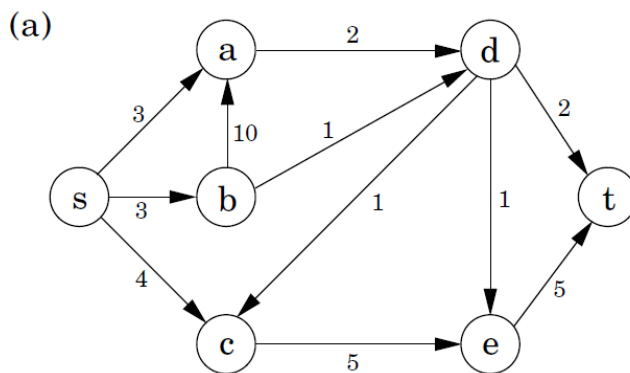
11 for nonnegativity

11 for capacity

5 for flow conservation

11 - # edges, 5 - # of intermediate nodes.

(a) A network with edge capacities. (b) A *flow* in the network.



So the problem is a generic LP one! But, max-flow algorithm exists independently.

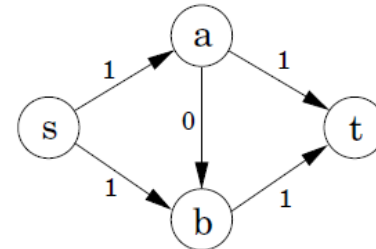
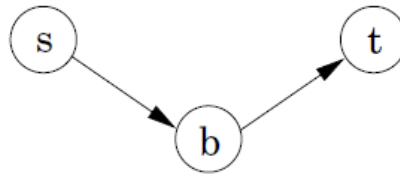
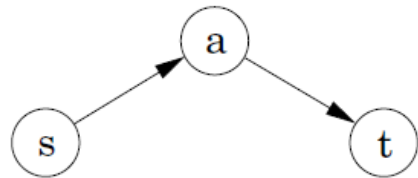
Start with zero flow.

Repeat: choose an appropriate path from s to t ,
and increase flow along the edges
of this path as much as possible.

Basically, apply a path finding algorithm (like DFS). Choose the path and send quantity through the path as much allowed by its bottleneck/s. Reduce the edge capacities by the corresponding current flows (remove the bottleneck edges), and then again repeat the above. Guaranteed to stop!

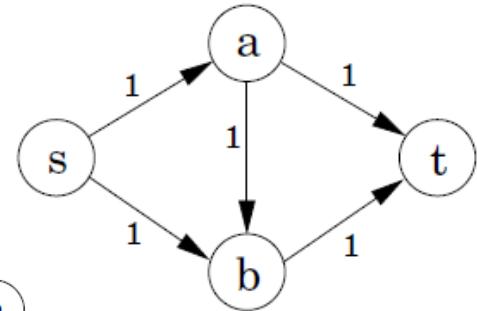
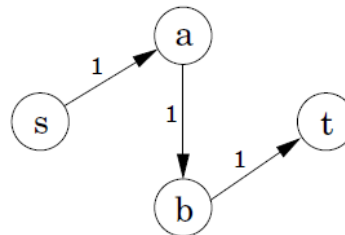
$O(m(m+n))$, But optimality is not guaranteed!

Start



End

What if starting & ending at:



We can get around this issue! **Ford-Fulkerson Algorithm**

– Cancel existing Flow

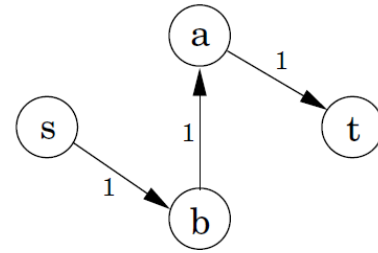
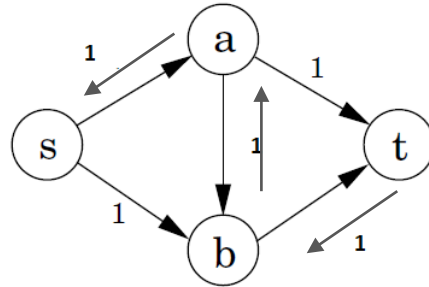
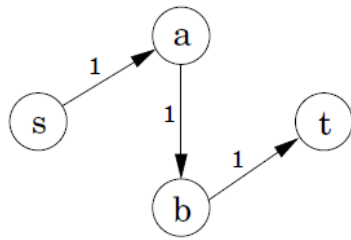
For every edge (u,v) in a chosen s - t path of the flow at an iteration an edge (v,u) is introduced with zero capacity & negative flow. Then, all the positive and negative flows are subtracted from their corresponding edge capacities to get the available capacities for the next iteration.

The network thus formed in every iteration is called the *residual network*.

$$G^f = (V, E^f)$$
$$c^f: \begin{cases} c_{uv} - f_{uv} & \text{if } (u,v) \in E \text{ and } f_{uv} < c_{uv} \\ 0 - (-f_{uv}) \leftarrow f_{vu} \text{ or } c_{vu} - (-f_{uv}) & \text{for } (v,u) \in E \end{cases}$$

↗ >0

- If an edge already existed in the opposite direction, that can easily be incorporated in the framework.
- The s - t path finding, say using linear-time DFS, happens on the residual network.

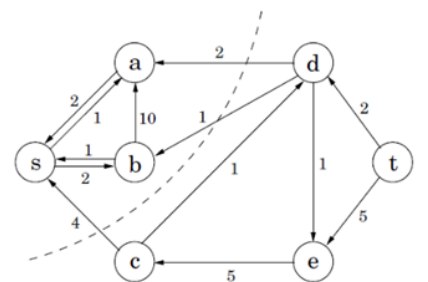
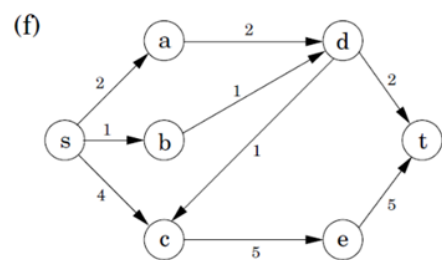
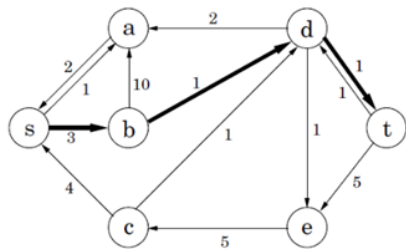
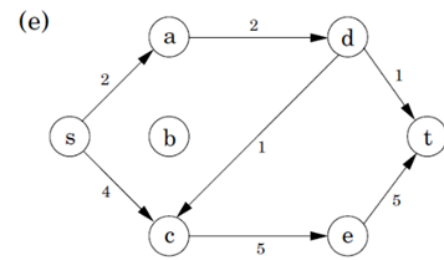
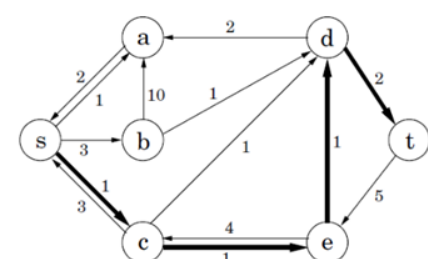
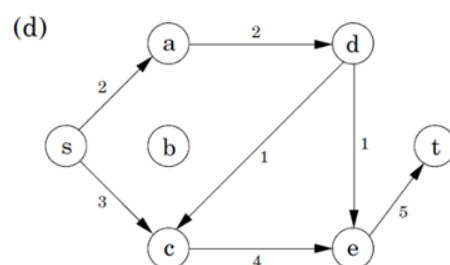
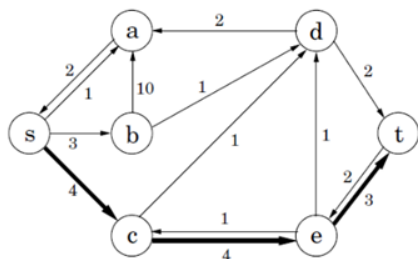
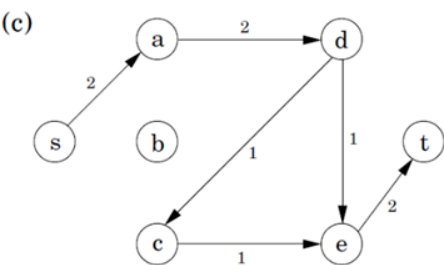
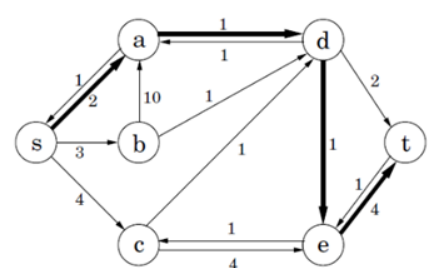
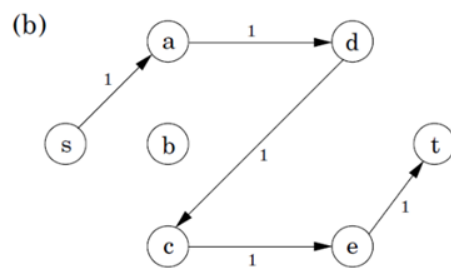
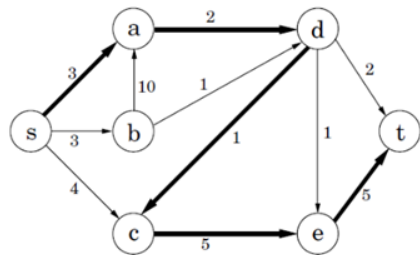
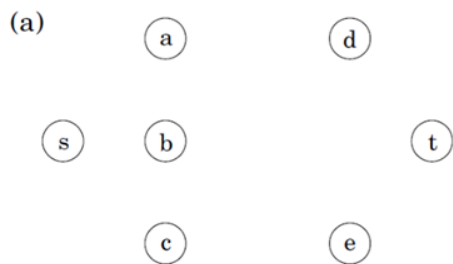


Will yield the same result as before!

The flow is updated by adding the corresponding partial weights in the selected s - t path to the current flow.

$$(u, v): f_{uv} + b \rightarrow f_{uv} \quad \text{or} \quad \max(f_{vu} - b, 0) \rightarrow f_{vu} \Rightarrow \max(b - f_{vu}, 0) \rightarrow f_{uv}$$

An example: Current Flow on the left, Residual Graph on the right...

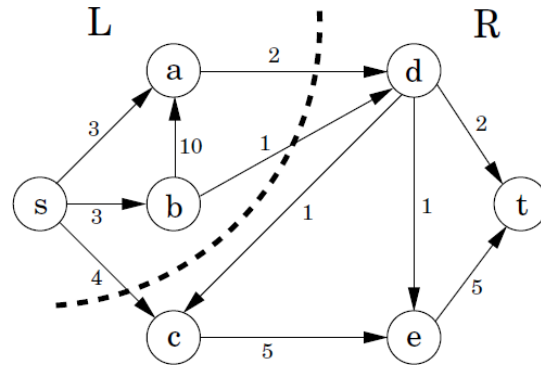


A certificate of optimality

Cut: $L = \{s, a, b\}$ $R = \{c, d, e, t\}$

(s, t) -cut

An (s, t) -cut partitions the vertices into two disjoint groups L and R such that s is in L and t is in R . Its capacity is the total capacity of the edges from L to R , and is an upper bound on any flow.



Tight upper bound: $L = \{s, a, b\}$ and $R = \{c, d, e, t\}$ (7)

Loose upper bound: $(\{s, b, c\}, \{a, d, e, t\})$ (19)

maximum flow

Pick any flow f and any (s, t) -cut (L, R) . Then $\text{size}(f) \leq \text{capacity}(L, R)$.

Max-flow min-cut theorem *The size of the maximum flow in a network equals the capacity of the smallest (s, t) -cut.*

This leads to the proof of correctness as well!

- In the original graph, we have seen that minimum cut capacity gives the maximum flow size.
- *[loose*] The Ford-Fulkerson algorithm **converges** as flows into the sink increases with iteration (cancel flows useless here), leaving no incoming capacity at some point.*

When the Ford-Fulkerson algorithm is applied:

- If the algorithm has achieved maximum flow size, no 'augmenting' path will be there in the residual graph to increase flow size.
- *Question:* If the algorithm stops due to 'augmenting' path being unavailable in the residual graph, does that mean the algorithm has achieved maximum flow size?

* It is enough to prove it results in the cut with minimum capacity!

$f \rightarrow$ Final Flow (t is no longer reachable from s in the residual network)

$$R = V \setminus L$$

$L \rightarrow$ set of all nodes reachable from s

Any e, e' in original graph

So the (L, R) is an s - t cut.

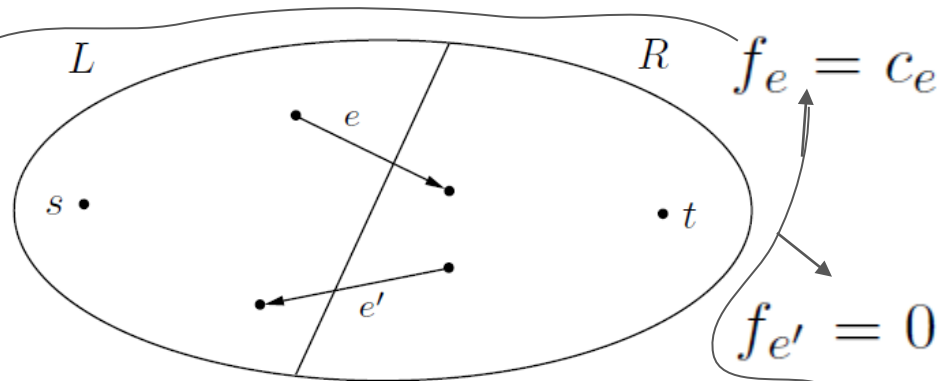
$\text{capacity}(L, R)$ is the min

To prove: $[tight^*]$

$$\text{size}(f) = \text{capacity}(L, R)$$

In the residual graph:

- By definition, as one cannot reach from L to R , any edge from L to R must be full in capacity.
- Secondly, if there was any edge from R to L that contributed to the flow, there should be a corresponding 'reverse' edge from L to R ! This is not possible as one cannot reach from L to R .



Efficiency

Dinic's Implementation: $O(mn^2)$

Edmonds-Karp Algorithm: $O(m^2n)$

- DFS (or BFS) linear time ($m + n$)
- Iterations?
 - If edge weights are integer (fixed-point real values) with maximum value as U (otherwise may not terminate):
Each s - t path's bottleneck edge will have at least a weight value 1. t node can have at most $n-1$ neighbors and the edges to the neighbors can have the maximum value of U . So maximum possible iteration: $(n-1)U \geq \text{val}(f^*)$. You won't be iterating anyway beyond $\text{val}(f^*)$.
So time complexity: $O((m+n) \text{val}(f^*))$
 - Number of iterations can be restricted, if BFS is used to find paths with fewest edges. (Edmonds-Karp)